



National Technical University of Athens

School of Electrical and Computer Engineering

8th Semester

VLSI Digital Systems

4th Laboratory Exercise

Dimitris Vatalas

(Student ID: 03121093)

Aristotelis Georgios Dritsas

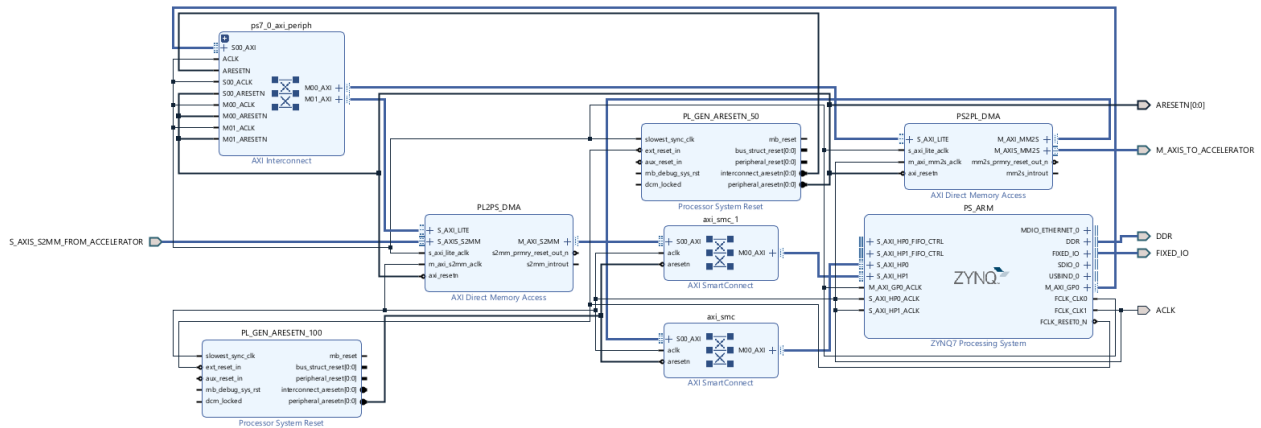
(Student ID: 03122127)

Athens, April 2026

Contents

1	System Block Diagram	2
2	Brief Explanation of the Main Structural Units of the Circuit	2
2.1	Calculator	2
2.2	Serial to Parallel	3
2.3	FSM	3
2.4	debayering_filter	3
3	FSM Diagram	4
4	FPGA Resource Utilization Analysis	4
5	Latency	5
6	Throughput	5

1 System Block Diagram



2 Brief Explanation of the Main Structural Units of the Circuit

2.1 Calculator

The calculator is responsible for computing the R , G , and B values for each pixel, using a 3×3 window of neighboring pixels (`din00` to `din22`) as input, as indicated in the assignment statement.

The generic parameter N is defined to determine the dimensions of the image to be processed. The circuit inputs include the control signals `clk`, `rst_n`, and `valid_in`, as well as the pixel data signals `dx`. Finally, the outputs consist of the computed R , G , and B channels, together with the `valid_out` signal.

In practice, the output computation is performed by an internal mechanism that identifies the type of each pixel (GREEN1, GREEN2, BLUE, or RED). Depending on the pixel type, the appropriate output calculation is performed according to the theory of the GBRG-type debayering filter.

Finally, regarding the correct handling of `valid_out`, when `valid_in` is equal to 1 during a clock cycle (and the circuit is not in the reset state, i.e., $rst_n = 0'$), we know that the processing during this cycle is valid, and therefore `valid_out` is also set to 1. Conversely, when `valid_in` is 0, no valid computation is performed, and `valid_out` is set to 0 accordingly.

2.2 Serial to Parallel

In this exercise, the data stream is serial. However, due to the nature of the debayering operation, appropriate data management is required so that the calculator receives the correct pixels at the correct time. This is where the `serial_to_parallel` module is introduced, which converts the serial data stream into a parallel representation suitable for processing. The implementation uses three FIFO queues, arranged as indicated in the corresponding lecture slides, while the 3×3 data window required by the calculator is essentially implemented using flip-flops.

During each clock cycle, the new pixel is inserted into the first FIFO and is then sequentially propagated through the following FIFOs. In this way, the FIFO outputs are used to form a 3×3 window. This window constitutes the valid input to the calculator, as described previously.

The entire process is synchronized using a pixel position counter, while the output is considered valid only after the first complete 3×3 processing window has been formed.

The generic parameter N is defined to determine the dimensions of the image to be processed. The circuit inputs include the signals `clk`, `rst_n`, and `valid_in`, as well as the incoming pixel stream `din`. The outputs consist of the elements of the 3×3 neighborhood (`dout00`–`dout22`), which feed the calculator, together with the `valid_out` signal, which indicates when the generated window is valid for processing.

2.3 FSM

This module implements a Finite State Machine (FSM) that manages the image processing flow, from the reception of the input data to the completion of the output. Its operation is based on an internal counter and a sequence of control states that determine the progress of the system.

The generic parameter N is defined to determine the dimensions of the image ($N \times N$). The circuit inputs include the signals `clk`, `rst_n`, `valid_in`, and `new_image`, while the outputs consist of the enable signal `en`, the counter `counter`, and the `image_finished` signal, which indicates the completion of the processing operation.

More specifically, the FSM consists of five states: `IDLE`, `RECEIVE`, `OUTPUTING`, `SHUTTING_DOWN`, and `DONE`. Initially, the FSM remains in the `IDLE` state until a new image is indicated, at which point the processing procedure begins. It then enters the `RECEIVE` state, where the $N \cdot N$ pixels are received while the `counter` is incremented whenever `valid_in` is active. Next, in the `OUTPUTING` state, the reception of new data stops, while the necessary computations continue for the data that are still stored in the FIFOs. The `SHUTTING_DOWN` state essentially provides one additional clock cycle of delay, ensuring that `image_finished` is asserted at the appropriate time, before transitioning to the final `DONE` state, which signals the completion of the process.

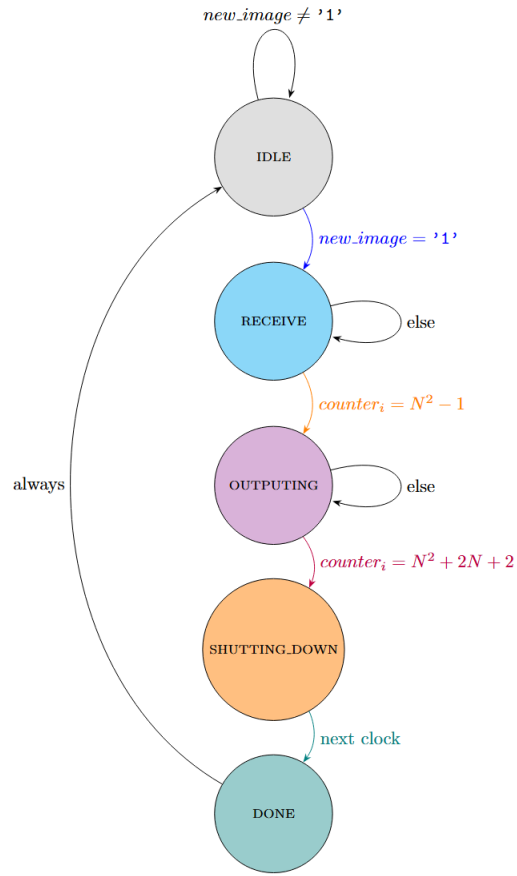
The `en` signal is activated only during the `RECEIVE` and `OUTPUTING` phases, while `image_finished` is set to '1' only when the FSM completes its cycle and reaches the `DONE` state.

2.4 debayering_filter

This module constitutes the top-level module of the debayering filter system. It does not contain any additional processing logic; instead, it serves exclusively as the interconnection layer between the individual submodules presented previously.

The generic parameter N is defined to determine the dimensions of the image to be processed ($N \times N$). The circuit inputs include the signals `clk`, `rst_n`, `new_image`, and `valid_in`, as well as the incoming pixel stream `pixel`. Correspondingly, the outputs consist of the color channels R , G , and B , the `valid_out` signal, and the `image_finished` signal, which indicates the completion of the processing operation.

3 FSM Diagram



4 FPGA Resource Utilization Analysis

- $N = 64$

Resource	Utilization	Available	Utilization %
LUT	3559	17600	20.22
LUTRAM	439	6000	7.32
FF	5196	35200	14.76
BRAM	12.50	60	20.83

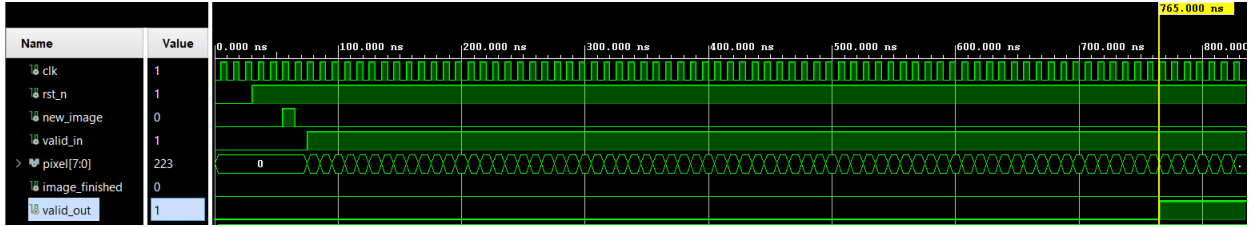
- $N = 128$

Resource	Utilization	Available	Utilization %
LUT	3571	17600	20.29
LUTRAM	438	6000	7.30
FF	5208	35200	14.80
BRAM	12.50	60	20.83

As expected, we observe that increasing the image size results in an increase in resource utilization. However, an interesting observation is that, due to the design optimization techniques applied during implementation, the increase is not as significant as might be expected when doubling the size of the three FIFOs. Instead, we observe relatively stable resource utilization despite the doubling of the size of the three FIFOs.

5 Latency

By running a behavioral simulation in order to determine the delay from the moment the first input pixel arrives until the first valid output is produced (i.e., the latency), we obtain the following result:



We observe that the first `valid_out` occurs at time 765 ns, while the first `valid_in` occurred at 75 ns. Given that the clock period is $T = 10$ ns, this corresponds to a latency of 68 clock cycles.

The above result could also be derived without running a simulation, since the following line is present in our code:

```
valid_out <= '1' when (counter_us > 2*N + 3 and en = '1') else '0';
```

Therefore, since the counter starts from 0 and for $N = 32$ we have:

$$2N + 3 = 2 \cdot 32 + 3 = 67$$

Thus, `valid_out` is activated when `counter_us` is 67, i.e., during the 68th clock cycle.

6 Throughput

Finally, regarding the throughput, we observe that after the initial latency, as shown in the following simulation snapshot, a valid result is produced during every clock cycle. Therefore, the throughput is equal to 1 result per clock cycle.

